MCP Servers

MCP SERVER UNIT 01
MCP SERVER - SKYNET OS v4.2
THE TERMINATOR THEME
MCP SERVER UNIT 01
GLOBAL ACCESS

Final Presentations

- **Final presentation schedules are done**
- **Schedule can be found on course website on Final Project page:**
  - https://ysghysgh.github.io/MGT_575_teams/

What is MCP?

- Model Context Protocol (MCP)
- An open standard for AI connectivity
- The bridge between agents (clients) and data (servers)
- Enables AI agents to easily access files, databases, and APIs securely

Benefits of MCP Servers

- Interoperability – write an MCP server once, and any AI agent can use it
- Freshness – AI gets live data from the source, not training memory
- Agentic action – agents can execute tasks
  - Summarize my last 5 Gmail emails
  - Play a Lord of the Rings playlist on Spotify
  - Make a Canva slide deck of this code repo

Origin Story

ANTHROP\C

Model Context Protocol

- Created in late 2024 by Anthropic
- Goal: stop “siloed” AI integrations
- Currently an open standard managed by the Linux Foundation

Popular MCP Servers

- Gmail – read, search, draft, send emails
- Google Calendar – schedule events
- Slack – channels, threads, search
- GitHub – repos, issues, pull requests
- Stripe – customers, charges, subscriptions
- Spotify – search, playback, playlists
- Canva – designs for slides, etc
- Zapier – connect to hundreds of SaaS tools (costs money)

Connecting to an MCP Server

- Cursor (the “client”) must know how to reach the MCP server
- Configuration information needed by Cursor
  - Server name (you can choose this)
  - Command to run the server (npx, node, python)
  - Arguments needed to run server, like its entry point
  - Environment variables, like API keys
- Once you connect, Cursor will have access to all tools in the MCP server
- All server config data sits in your mcp.json file

mcp.json

```
{
  "mcpServers": {
    "gmail": {
      "command": "npx",
      "args": [
        "-y",
        "@gongrzhe/server-gmail-autoauth-mcp"
      ]
    },
    "PowerPoint": {
      "command": "uvx",
      "args": [
        "--from",
        "office-powerpoint-mcp-server",
        "ppt_mcp_server.exe"
      ]
    },
    "Canva": {
      "command": "npx",
      "args": [
        "-y",
        "mcp-remote@latest",
        "https://mcp.canva.com/mcp"
      ]
    },
    "spotify": {
      "command": "npx",
      "args": [
        "-y",
        "@tbrgeek/spotify-mcp-server"
      ],
      "env": {
```

Adding an MCP Server to Cursor

1. Go to Cursor Settings -> Tools & MCPs
2. Add a server block in the mcp.json file (this is a JSON object like the one show below)
3. Input any env variables you need (API keys)
   - WARNING: Don’t commit your server with API keys in the repo
4. Use Reload MCP or restart Cursor to spawn fresh server processes
5. Enable the server with toggle button in Tools & MCPs
   - You will be able to see the tools the server has

```
{
  "mcpServers": {
    "my-demo": {
      "command": "node",
      "args": ["C:/path/to/your/server/dist/index.js"],
      "env": { "SOME_API_KEY": "sk-..." }
    }
  }
}
```

Example: Connecting to Gmail MCP Server

- Update mcp.json so it include Gmail MCP server info

```
{
  "mcpServers": {
    "gmail": {
      "command": "npx",
      "args": [
        "-y",
        "@gongrzhe/server-gmail-autoauth-mcp"
      ]
    }
  }
}
```

- Reload MCP or restart Cursor
  - Cursor will run the command with arguments
- First time authorization: browser should pop up asking for your Gmail login information
- Toggle server on in Cursor Tools & MCPs menu

Use Gmail MCP Server

- **Ask Cursor to do Gmail things**

send a gmail to Yen-Shao chen in Chinese telling a funny joke about chicken

Explored 2 files, 1 search

Ran Search Emails in gmail

Thought for 9s

Found their address in your mailbox: `yen-shao.chen@yale.edu`. Sending a short, lighthearted message in Traditional Chinese (matching the name style).

Ran Send Email in gmail

Sent. Gmail went to yen-shao.chen@yale.edu (from your mailbox search for "Chen, Yen-Shao").

Subject: 分享一則關於雞的冷笑話 🐔

Body (Chinese): Brief greeting to **蔡韶老師**, a short pun-style joke about chicks and **代數** ("algebra" vs. "代雞"), a self-deprecating line that it's okay if it's not funny, and a sign-off wishing smooth research and a good mood.

Message id from the tool: `19da7d15ebeae832`.

Your search also turned up c.yenshao@gmail.com (e.g. Overleaf). If you'd rather use that address next time, say which one to use.

...

send him a follow up email in English saying im testing the MCP server for gmail. and Im tauhid zaman is Phd adivsor. be humorous

Thought briefly

Ran Send Email in gmail

Sent a follow-up to yen-shao.chen@yale.edu.

Subject: `RE: chicken diplomacy — now with 100% more MCP`

Tone: Light English note that you're **testing a Gmail MCP integration**, you identify as **Tauhid Zaman, his PhD advisor**, with jokes about experiments, coffee vs. automation, and "no chickens harmed (several jokes were)."

Gmail message id: `19da7d1e9092cbcb`.

Trickier Example: Spotify MCP

- **Create a Spotify app:**
  - **Go to https://developer.spotify.com/dashboard**
  - **Create App with name and description**
  - **Redirect URL needs to be http://127.0.0.1:8888/callback**
- **Copy app env variables**
  - **SPOTIFY_CLIENT_ID**
  - **SPOTIFY_CLIENT_SECRET**
  - **SPOTIFY_REDIRECT_URI (this is the redirect URL shown above)**
- **Scopes – in app dashboard, enable all features (why not)**
- **Make sure you have a Premium subscription to play songs**

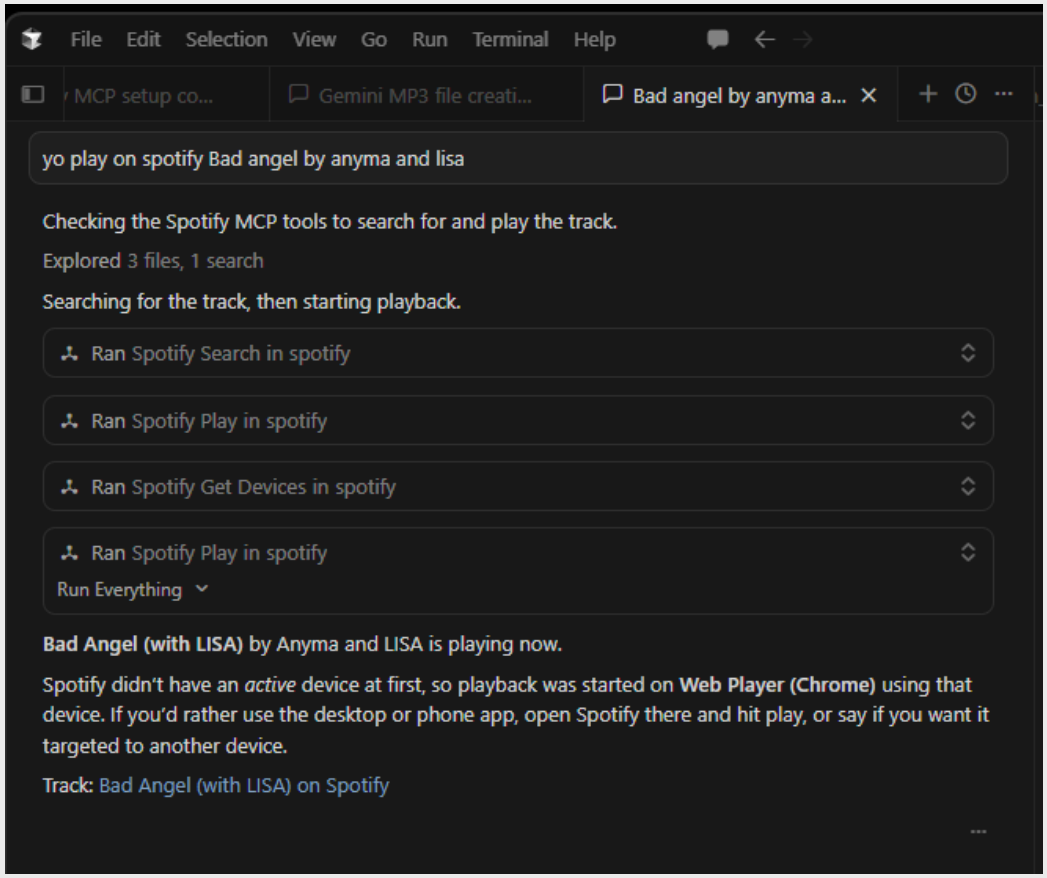
Trickier Example: Spotify MCP

- **Now you can update mcp.json in Cursor**

```
{
  "mcpServers": {
    "spotify": {
      "command": "npx",
      "args": [
        "-y",
        "@tbrgeek/spotify-mcp-server"
      ],
      "env": {
        "SPOTIFY_CLIENT_ID": "YOUR_SPOTIFY_CLIENT_ID",
        "SPOTIFY_CLIENT_SECRET": "YOUR_SPOTIFY_CLIENT_SECRET",
        "SPOTIFY_REDIRECT_URI": "http://127.0.0.1:8888/callback"
      }
    }
  }
}
```

Using Spotify MCP Server

- **Make sure to open the Spotify website and login**
- **Then ask Cursor to do Spotify things**



Building an MCP Server

- Define resources: files, docs, etc.
- Define tools: functions for AI to call
- Tell Cursor to build it for you
  - Server codes resides in a single code file

Python MCP Server

- **Example Python MCP server that returns stock prices**
- **Server file: stock_mcp.py**

```python
"""Minimal MCP server: one tool, yfinance, stdio (official FastMCP)."""
from mcp.server.fastmcp import FastMCP
import yfinance as yf

mcp = FastMCP("Yale-SOM-Stocks")

@mcp.tool()
def stock_price(symbol: str) -> str:
    """Latest available daily close for a ticker (e.g. AAPL, MSFT)."""
    sym = symbol.strip().upper().replace("-", ".")  # e.g. BRK-B -> BRK.B
    t = yf.Ticker(sym)
    hist = t.history(period="5d")
    if hist.empty:
        return f"No price data for {sym}"
    last = float(hist["Close"].iloc[-1])
    return f"{sym} last close: {last:.2f} (via yfinance)"

if __name__ == "__main__":
    mcp.run()  # stdio — Cursor spawns this process
```

Adding Server to mcp.json

- **Add this block of code to mcp.json**

```
"yale-stocks": {
  "command": "python",
  "args": [
    "C:/Users/you/code/stock_mcp.py"
  ]
}
```

Coding Session

- Connect existing MCP servers to Cursor
- Create a simple MCP server in Cursor